

Metodología FDD

Feature Driven Development

Bryan Maximiliano Dzul Perez

Materia: Proyecto de una Plataforma Virtual
Prof. Mario Alonso Segovia Gutiérrez
29/05/2026

¿Qué es FDD?

Definición y Objetivo

El **Feature Driven Development (FDD)** es una metodología ágil diseñada para organizar el Desarrollo de software enfocándose en características tangibles y valoradas por el cliente.

Su objetivo principal es entregar software funcional de forma repetible, oportuna y bajo procesos de ingeniería estrictos, evitando la ambigüedad.

¿Qué busca resolver?

- Superar la complejidad de proyectos masivos que las metodologías tradicionales no podían manejar.
- Evitar la falta de visibilidad del progreso en el desarrollo
- Minimizar el riesgo de integración dividiendo el sistema en módulos manejables
- Establecer un puente claro entre la necesidad del negocio y el código técnico.

Contexto y Origen

- Creada en 1997 por Jeff De Luca y refinada por Peter Coad.
- **El problema:** Un gigantesco proyecto de software de 15 meses y 50 personas para un gran **banco de Singapur** estaba al borde del fracaso por su alta complejidad.
- **La Solución:** De Luca diseñó un enfoque de 5 pasos centrado en iteraciones cortas (modelo, listas, planificación, diseño y construcción)
- **El Impacto:** Permitió a equipos grandes ser adaptables, respondiendo a requerimientos cambiantes.



Funcionamiento: 5 Procesos



Desarrollar un Modelo Global: Se crea un modelo de dominio colaborativo para entender el problema central del negocio.



Construir la Lista de Features: Se agrupan los requerimientos en características pequeñas que aporten valor al cliente (acción, resultado, objeto)



Planear por Función: Se priorizan y asignan las features a los programadores evaluando, riesgos, carga de trabajo y dependencias.



Diseñar por Función: El programador jefe revisa y diseña el paquete técnico de la funcionalidad junto al equipo.



Construir por Función: Se codifica, se prueba y se inspecciona. Si aprueba, se integra al sistema principal inmediatamente.

Feature Driven Development (FDD)

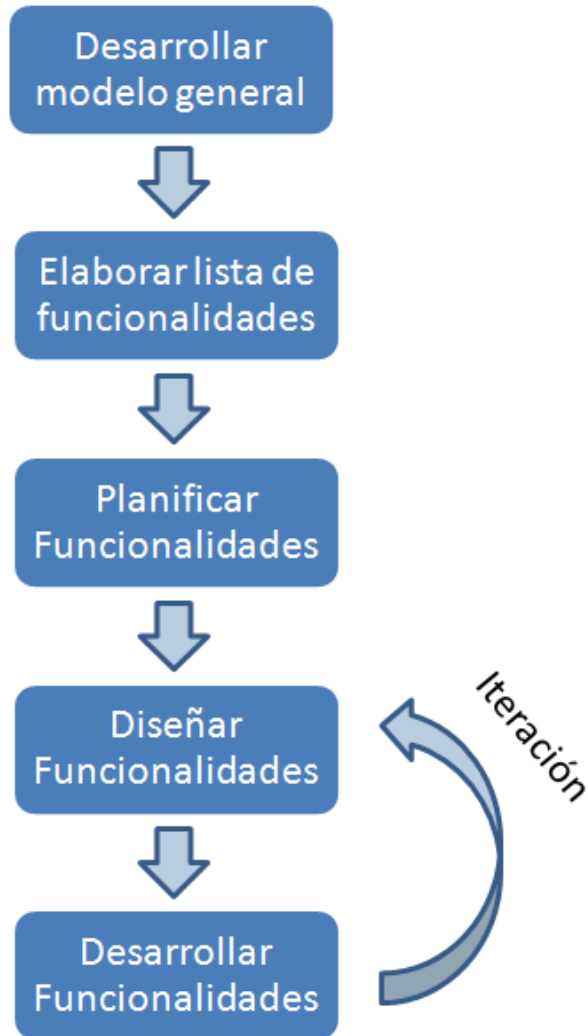


Imagen elaborada por oficinaproyectossoftware.blogspot.com

Organización y Roles del Equipo



Project Manager

Administra el proyecto, presupuestos y cronogramas. Es el puente administrativo principal



Chief Architect

Lidera el modelado del dominio. Define la arquitectura global y asegura la integridad técnica.



Chief Programmer

Lidera equipos de funcionalidad temporales. Coordina diseño, código y revisiones técnicos.



Class Owner

Desarrolladores responsables de la consistencia, rendimiento y código de clases específicas.

Gestión de tareas y avance

14

Días máximo

(Tiempo límite por feature)



Control Estricto y Priorización

En FDD, el trabajo se divide en **features minúsculas**. Si una funcionalidad toma más de dos semanas, debe subdividirse. Esto garantiza un flujo constante.

- **Avance por Hitos:** Cada feature pasa por hitos medibles (Diseño, Desarrolla, Pruebas).
- **Prioridad:** Basada en valor para el negocio y dependencias técnicas.
- **Herramientas:** Uso de Git, Jira/Trello y repositorio de control continuo.

Análisis: Ventajas y Desventajas

👍 Ventajas

Escalabilidad: Crece fácilmente desde equipos pequeños a organizaciones masivas.

Visibilidad: Los 6 hitos por feature permiten medir el avance real y detectar retrasos al instante.

Menos Reuniones: Se basa en documentación sólida y modelado, reduciendo la sobrecarga de juntas.

Calidad: Inspecciones de código estrictas previenen errores graves a futuro.

👎 Desventajas

Curva de Aprendizaje: Require una alta experiencia en modelado y arquitectura de sistemas.

Rigidez ante Cambios Mayores: Un cambio drástico en los requerimientos puede afectar todo el modelado de dominio.

Poco Amigables para Startups: Su estructura jerárquica no es ideal para equipos menores de 10 personas.



¿Cuándo aplicar FDD?

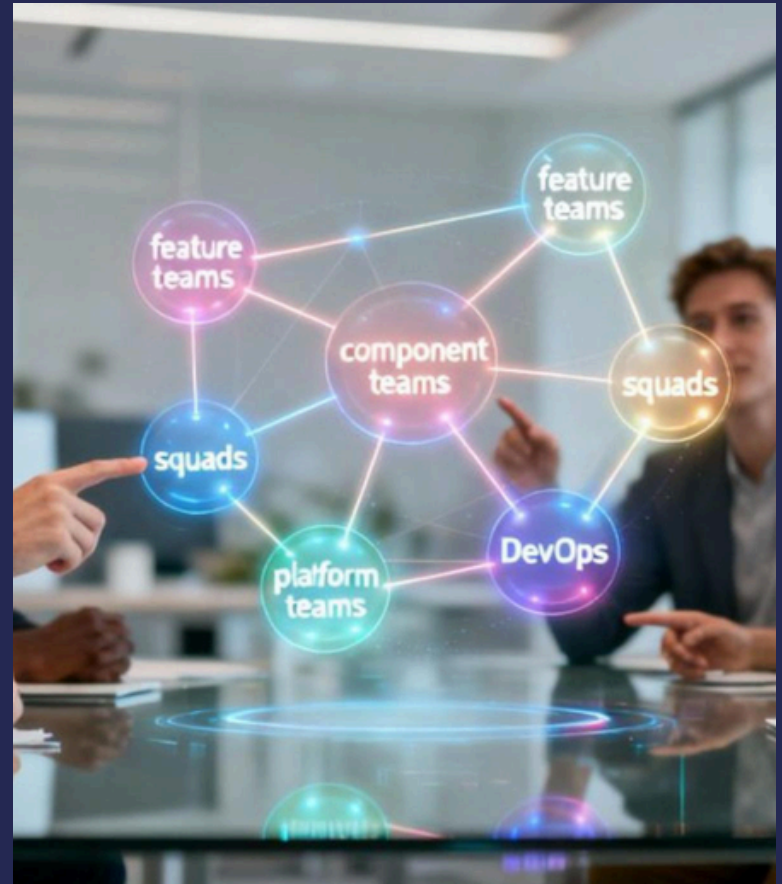
Entornos Empresariales Complejos

FDD brilla en proyectos donde el dominio del problema es vasto y el equipo de desarrollo supera los 30-50 integrantes. Su estructura lo hace perfecto para industrias con baja tolerancia al error.

Casos ideales:

Sistemas Bancarios y Financieros.

- Software Hospitalario y Gestión médica.
- Grandes ERPs Corporativos
- Modernización de sistemas heredados (Legacy)



Comparativa

Dimensión	FDD	Melé	Kanban
Organización del trabajo	Features(de una a dos semanas cada	Sprint (uno a cuatro semanas)	Flujo continuo
Enfoque de planificación	Modelado inicial, lista de funciones	Planificación del sprint, lista de tareas pendientes	Justo a tiempo
Estructura del equipo	Jerárquico con roles definidos	Autoorganización	Flexible
Lo mejor para	Equipos grandes, dominios complejos	Cambio rápido, equipos pequeños	Mantenimiento y soporte

Ejemplo: Plataforma E-commerce

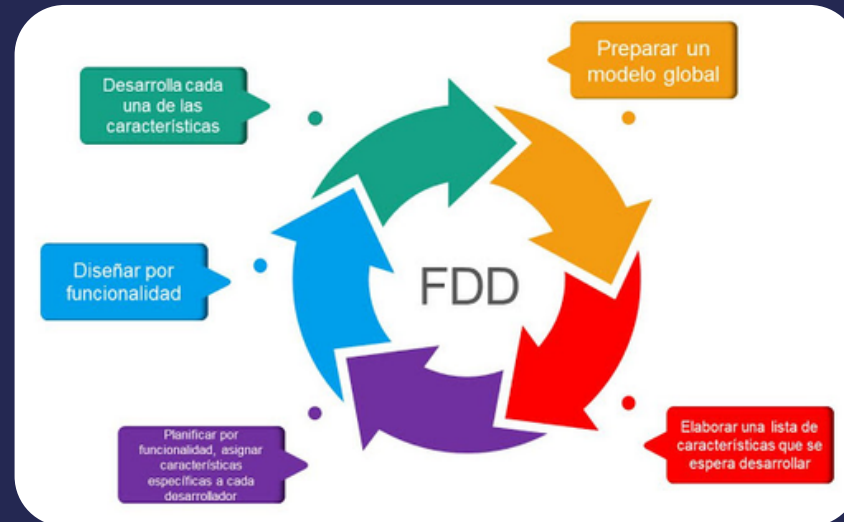
Feature: “Añadir al Carrito”

En lugar de construir todo el módulo de ventas a la vez, FDD aísla esta función:

- **Experto de Dominio:** Define reglas (ej: validar inventario antes de añadir)
- **Chief Architect:** Asegura la conexión de esta feature con la Base de Datos y Sesiones de Usuario.
- **Chief Programmer:** Lidera un equipo temporal para desarrollar la función en menos de 10 días.
- **Class Owners:** Codifican las clases ‘Producto’, ‘Carrito’ y ‘Sesión’ en paralelo, integrando sus partes al final de la iteración.



Gracias



Fuentes:

<https://www.lucidchart.com/blog/es/utilizar-el-desarrollo-basado-en-funciones>

<https://monday.com/blog/rnd/feature-driven-development-fdd/>

<https://www.pmoinformatica.com/2012/11/5-preguntas-y-respuestas-sobre-el.html>